

gemstone-rs BridgeRoot and Object Mapping

A first MagLev-style bridge-root mapping layer over OOPs



Generated from the Markdown sources in `gemstone-rs/docs`.

Contents

BridgeRoot and Object Mapping

What Is Supported

Rust Example

Typed Rust Struct Mapping

Derive-Based Mapping

Nested Read-Back

Key Policy

Transactions and Identity

Relationship-Shaped Payloads

Comparison With MagLev/GBS

Codegen Support

Explorer and VS Code Support

Current Boundaries

BridgeRoot and Object Mapping

`gemstone-rs` now has a first object-mapping layer over the lower-level `Oop` API.

This is intentionally smaller than MagLev/GBS object mapping. It gives Rust code a bridge-root dictionary, typed Rust payload mapping, nested read-back, and generated mapping support while keeping direct OOP access available.

What Is Supported

The initial mapping layer supports:

- `Session::bridge_root()`
- `Session::bridge_root_named(name)`
- `BridgeRoot::put`
- `BridgeRoot::put_mapped`
- `BridgeRoot::get_oop`
- `BridgeRoot::get_value`
- `BridgeRoot::get_string`
- `BridgeRoot::get_smallint`
- `BridgeRoot::get_bool`
- `BridgeRoot::get_dictionary`
- `BridgeRoot::get_mapped`
- `BridgeRoot::remove`
- `BridgeRoot::commit`
- `BridgeRoot::commit_with_retry`
- `BridgeRoot::transaction`
- `BridgeRoot::keys`
- `BridgeRoot::contains_key`
- `BridgeDictionary::at_oop`
- `BridgeDictionary::at_value`
- `BridgeDictionary::at_string`
- `BridgeDictionary::at_smallint`
- `BridgeDictionary::at_bool`
- `BridgeDictionary::at_dictionary`
- `BridgeDictionary::at_mapped`
- `BridgeDictionary::at_vec`
- `BridgeDictionary::contains_key`
- `BridgeDictionary::keys`
- `BridgeKey`
- `BridgeKeyType::String`
- `BridgeKeyType::Symbol`

- BridgeMapped
- #[derive(BridgeMapped)]
- BridgeFieldRead
- BridgeFieldWrite
- BridgeValue::Nil
- BridgeValue::Bool
- BridgeValue::SmallInt
- BridgeValue::String
- BridgeValue::Symbol
- BridgeValue::Oop
- BridgeValue::Dictionary
- BridgeValue::KeyedDictionary
- BridgeValue::Array
- session-local OOP identity ids with Session::identity_for_oop

The default root is a GemStone Dictionary stored in UserGlobals under #GemStoneRsBridgeRoot.

Rust Example

```
use gemstone_rs::{BridgeValue, Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env())?;

    let payload = BridgeValue::dictionary([
        ("name".to_string(), BridgeValue::from("Tariq")),
        ("amount".to_string(), BridgeValue::from(100_i64)),
        ("currency".to_string(), BridgeValue::from("GBP")),
    ]);

    let mut bridge_root = session.bridge_root()?;
    let payload_oop = bridge_root.put("MyTestDict", payload)?;
    let stored = bridge_root.get_oop("MyTestDict")?;
    assert_eq!(payload_oop, stored);

    bridge_root.commit()?;
    Ok(())
}
```

Run the example:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
```

Typed Rust Struct Mapping

For application code, implement `BridgeMapped` on a plain Rust type. The trait keeps the mapping explicit and reviewable while removing repeated dictionary field reads from application code.

```
use gemstone_rs::{BridgeDictionary, BridgeMapped, BridgeValue, Config, Session};

#[derive(Debug, Eq, PartialEq)]
struct BookingDraft {
    name: String,
    amount: i64,
    currency: String,
}

impl BridgeMapped for BookingDraft {
    fn to_bridge_value(&self) -> BridgeValue {
        BridgeValue::dictionary([
            ("name".to_string(), BridgeValue::from(self.name.clone())),
            ("amount".to_string(), BridgeValue::from(self.amount)),
            ("currency".to_string(), BridgeValue::from(self.currency.clone())),
        ])
    }

    fn from_bridge_dictionary(dictionary: &mut BridgeDictionary<'_>) ->
gemstone_rs::Result<Self> {
        Ok(Self {
            name: dictionary.at_string("name")?,
            amount: dictionary.at_smallint("amount")?,
            currency: dictionary.at_string("currency")?,
        })
    }
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env())?;
    let mut bridge_root = session.bridge_root()?;

    let draft = BookingDraft {
        name: "Tariq".to_string(),
        amount: 100,
        currency: "GBP".to_string(),
    };

    bridge_root.put_mapped("MyTestDict", &draft)?;
    let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
    assert_eq!(loaded, draft);

    bridge_root.commit()?;
```

```
    Ok(())  
}
```

The checked-in example uses this pattern:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot  
MyTestDict OOP: <number>  
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP" }
```

Derive-Based Mapping

For normal Rust structs, prefer `#[derive(BridgeMapped)]`. The derive writes a `BridgeMapped` implementation that stores fields in a GemStone dictionary and reads them back into Rust.

```
use gemstone_rs::{BridgeMapped, Config, Session};  
  
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]  
struct CustomerDraft {  
    #[bridge(key_type = "Symbol")]  
    name: String,  
}  
  
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]  
struct BookingDraft {  
    #[bridge(key = "amount", key_type = "Symbol")]  
    amount: i64,  
    customer: CustomerDraft,  
    tags: Vec<String>,  
}  
  
fn main() -> gemstone_rs::Result<()> {  
    let mut session = Session::login(Config::from_env())?;  
    let mut bridge_root = session.bridge_root()?;  
  
    let draft = BookingDraft {  
        amount: 100,  
        customer: CustomerDraft {  
            name: "Tariq".to_string(),  
        },  
        tags: vec!["priority".to_string(), "demo".to_string()],  
        note: None,  
    };  
  
    bridge_root.put_mapped("DerivedBookingDraft", &draft)?;
```

```

let loaded: BookingDraft = bridge_root.get_mapped("DerivedBookingDraft")?;
assert_eq!(loaded, draft);

bridge_root.commit()?;
Ok(())
}

```

Run the checked-in live example:

```
cargo run -p gemstone-rs --example derive_mapping
```

Expected output includes:

```

derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"], note: None }
bridge root identity: <number>

```

Nested Read-Back

Nested mapped structs and arrays round-trip through the same bridge dictionary format:

Rust field	GemStone storage	Read-back helper
<code>String</code>	<code>String</code>	<code>BridgeFieldRead</code> / <code>at_string</code>
<code>i64</code>	<code>SmallInteger</code>	<code>BridgeFieldRead</code> / <code>at_smallint</code>
<code>bool</code>	<code>true</code> / <code>false</code>	<code>BridgeFieldRead</code> / <code>at_bool</code>
<code>Oop</code>	raw object reference	<code>BridgeFieldRead</code> / <code>at_oop</code>
another <code>BridgeMapped</code> struct	nested <code>Dictionary</code>	<code>BridgeDictionary::at_mapped</code>
<code>Vec<T></code>	<code>Array</code>	<code>BridgeDictionary::at_vec</code>
<code>Option<T></code>	value, <code>nil</code> , or missing key	<code>BridgeFieldRead</code>

This lets a Rust payload keep normal nested shape:

```

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    customer: CustomerDraft,
    tags: Vec<String>,
}

```

```
    note: Option<String>,  
  }
```

Optional fields are useful when a BridgeRoot dictionary evolves over time. `None` writes as `GemStone nil`; read-back returns `None` when the key is missing or when the stored value is `nil`. `Some(value)` reads through the inner field type, so `Option<CustomerDraft>` still reports nested mapping errors with the full field path.

When read-back fails, mapping errors include field context:

```
field amount expected GemStone value type SmallInt, got Oop 1234  
field booking.customer.name expected GemStone value type String, got OOP 1234  
field tags[2] expected GemStone value type String, got OOP 1234
```

Nested mapped read-back preserves the full field path, and array read-back reports the failing element index. Both are important when generated mappings read back nested payloads from a live stone. Lookup failures are also wrapped with the current path, so missing keys and invalid nested arrays point at `booking.items` or `booking.items[2]` instead of only returning a generic `GemStone` lookup error.

Key Policy

`GemStone` dictionaries often use either string keys or symbol keys. The mapping layer makes that explicit:

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]  
struct BookingDraft {  
    #[bridge(key = "amount", key_type = "Symbol")]  
    amount: i64,  
}
```

Manual code can use the same policy:

```
use gemstone_rs::BridgeKeyType;  
  
bridge_root.put_with_key_type("BookingDraft", BridgeKeyType::Symbol, "stored"?;  
let oop = bridge_root.get_oop_with_key_type("BookingDraft", BridgeKeyType::Symbol)?;
```

Use string keys when interoperating with JSON-like payloads and symbol keys when you want Smalltalk-style dictionary access.

Transactions and Identity

`BridgeRoot::transaction` commits on success and aborts on error:

```
bridge_root.transaction(|root| {
    root.put_mapped("BookingDraft", &draft)?;
    let loaded: BookingDraft = root.get_mapped("BookingDraft"?);
    assert_eq!(loaded, draft);
    Ok(())
})?;
```

For conflict-prone commits, use a bounded retry:

```
bridge_root.put_mapped("BookingDraft", &draft)?;
bridge_root.commit_with_retry(2)?;
```

`Session` also tracks a session-local identity id for OOPs it sees:

```
let oop = bridge_root.get_oop("BookingDraft"?);
let identity = bridge_root.identity_id();
println!("bridge root identity={identity}, payload oop={}", oop.raw());
```

That is not transparent object persistence. It is a stable in-session cache key for wrappers, inspectors, and explorer views.

Relationship-Shaped Payloads

Use nested structs and vectors when the Rust boundary needs a relationship-like shape but you still want explicit persistence. This keeps the GemStone side a plain dictionary graph and keeps the Rust side strongly typed:

```
use gemstone_rs::BridgeMapped;

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct CustomerDraft {
    name: String,
    email: String,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct LineItemDraft {
    sku: String,
    quantity: i64,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
```

```
struct BookingDraft {
    customer: CustomerDraft,
    items: Vec<LineItemDraft>,
    tags: Vec<String>,
    note: Option<String>,
}
```

A failed nested read reports the path that a user would naturally inspect:

```
booking.items[2].customer.name expected GemStone value type String, got OOP 1234
```

This is the practical middle ground between raw OOPs and transparent object persistence: relationships are visible in config and generated Rust source, and the low-level OOP API remains available for special cases.

Comparison With MagLev/GBS

The MagLev branch example uses:

```
session bridgeRoot at: #MyTestDict put: payload.
session commitTransactionOrSignalConflict
```

The closest current `gemstone-rs` shape is:

```
let mut bridge_root = session.bridge_root()?;
bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

For typed Rust structs:

```
bridge_root.put_mapped("MyTestDict", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
```

This is still explicit mapping, not transparent persistence. The benefit is that Rust teams can review every field mapping and still keep direct OOP access available when they need it.

Codegen Support

`gemstone-rs` codegen can emit `BridgeMapped` structs from config:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
```

```
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
field = BookingDraft.note | type=Option<String> | key=note
```

Generate a mapping config proposal from a live GemStone class:

```
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

For quick CLI inspection of the bridge root itself:

```
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge get BookingDraft --symbol
gemstone-rs bridge inspect BookingDraft --symbol
gemstone-rs bridge sample-config BookingDraft
```

Use `--root OtherBridgeRoot` when you intentionally use a non-default root dictionary, and `--key-type String|Symbol` when you want the key policy to be spelled out in scripts.

`bridge keys` lists each root key with its key OOP, class OOP, `printString`, and session-local identity id.

Run the live discovery example:

```
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated mapping example:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

This is the recommended path for repeated mappings because the generated source is checked in and reviewed like any other Rust code.

Explorer and VS Code Support

The local explorer exposes BridgeRoot-oriented endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s http://127.0.0.1:8787/api/bridge/keys
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/put?key=WorkbenchDraft&value=hello'
curl -s 'http://127.0.0.1:8787/api/bridge/remove?key=WorkbenchDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
```

```
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?
mapped=BookingDraft&class=Object'
```

The write endpoints require `gemstone-rs-explorer --allow-write`. Without that flag they return HTTP 403, matching the explorer's read-only default.

The VS Code workbench adds the same object-mapping workflow:

- GemStone RS: Generate Mapping Config
- GemStone RS: Preview BridgeRoot
- GemStone RS: List BridgeRoot Keys
- GemStone RS: Put BridgeRoot String
- GemStone RS: Remove BridgeRoot Key
- GemStone RS: Run Generated Mapping Example

The sidebar also shows these actions under `Codegen Config`.

Current Boundaries

The mapping layer is explicit, not transparent persistence. It does not yet make every GemStone object look like a native Rust object automatically. The current design is deliberately conservative:

- `Oops` remains visible and available.
- `Session` is still non-`Send` and non-`Sync`.
- writes happen only through explicit `put`, `put_mapped`, or generated code.
- the identity map is session-local and does not replace GemStone identity.

This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.